

Stage2 可微 ICCD 技术总结

第二阶段阶段性训练与分场景评估总结

更新时间：2026-07-09

1. 本轮目标

本轮目标不是直接把所有信号类型一次性强行训练到最好，而是按稳定性逐步推进：

- 1. 先从较简单、较稳定的信号开始，例如线性 chirp、二次多项式 chirp、三次多项式 chirp。
- 2. 如果简单场景下 IF-Net + 可微 ICCD 层能够稳定重构，再扩展到全部类型。
- 3. 对全部类型做分场景评估，找出哪些类型已经满足第二阶段继续推进的需要，哪些类型还需要单独优化。
- 4. 对困难类型尝试专家模型或专门配置，判断是否能通过更长训练解决。

2. 已完成的代码工作

本轮在第二阶段代码中补充了分场景评估和多种训练配置：

- 新增 `src/stage2_iccd/eval_scenarios.py`，用于对指定 checkpoint 按信号类型分别评估。
- 新增 `configs/separated_frozen.yaml`，用于简单分离场景的课程训练。
- 新增 `configs/all_multiexpert.yaml`，用于多 IF-Net 专家候选输入的全类型训练实验。
- 新增 `configs/local_jump_frozen.yaml`，用于 `local_jump` 专门训练。
- 新增 `configs/sinusoidal_frozen.yaml`，用于 `sinusoidal_fm` 专门训练。
- 修改 `model.py` 中的候选 IF 混合器：
 - 原先是全局可学习平均，容易把不同专家输出的 IF 曲线直接平均成不存在的轨迹。
 - 现在增加基于 ICCD 初步重构残差的动态候选权重，让每个样本根据重构误差选择更合适的候选。
- 修改训练日志，记录候选权重均值和候选选择温度。

3. 训练路径与主要结果

3.1 简单分离场景训练

训练对象：linear、quadratic、cubic。

使用配置：`configs/separated_frozen.yaml`

训练后 checkpoint: `stage2_iccd/runs/separated_frozen/latest.pt`

分场景评估结果：

场景	重构 SNR / dB	IF MAE / Hz
linear	23.40	2.38
quadratic	23.02	2.47
cubic	21.64	3.55

场景	重构 SNR / dB	IF MAE / Hz
aggregate	22.69	2.80

结论：简单场景训练是稳定的。说明第二阶段的可微 ICCD 层、alpha 学习、候选 IF 权重和小型 IF refinement head 都能正常参与训练，并且没有出现明显梯度或重构不稳定问题。

3.2 从简单场景扩展到全部类型

使用 separated_frozen 作为初始化，继续训练全部类型。

训练后 checkpoint: stage2_iccd/runs/all_from_separated/latest.pt

这是目前最稳的通用第二阶段 checkpoint。

分场景评估结果：

场景	重构 SNR / dB	IF MAE / Hz	判断
linear	21.18	3.26	可接受
quadratic	21.76	2.40	较好
cubic	21.52	3.22	可接受
sinusoidal_fm	17.67	5.58	偏弱
crossing	19.97	3.03	可接受
near_parallel	23.35	1.71	最稳定
local_jump	16.17	6.70	明显偏弱
tangent_or_overlap	21.11	3.13	可接受
aggregate	20.34	3.63	总体可用

结论：全类型一起推进是可行的，但不是所有类型同步受益。当前通用模型对 linear、quadratic、cubic、crossing、near_parallel、tangent_or_overlap 的表现已经比较稳定；主要短板集中在 local_jump 和 sinusoidal_fm。

4. 困难类型的补充实验

4.1 sinusoidal_fm 专门模型

训练后 checkpoint: stage2_iccd/runs/sinusoidal_frozen/latest.pt

评估结果：

场景	重构 SNR / dB	IF MAE / Hz
sinusoidal_fm	20.28	4.05

相对通用模型：

- 通用模型：17.67 dB, 5.58 Hz
- sinusoidal 专门模型：20.28 dB, 4.05 Hz

结论：sinusoidal_fm 可以通过专门训练明显改善。后续可以保留通用模型，也可以在路由器足够稳定后为 sinusoidal_fm 使用专门分支。

4.2 local_jump 专门模型

训练后 checkpoint: stage2_iccd/runs/local_jump_frozen/latest.pt

评估结果：

场景	重构 SNR / dB	IF MAE / Hz
local_jump	15.15	8.34

相对通用模型：

- 通用模型：16.17 dB, 6.70 Hz
- local_jump 专门模型：15.15 dB, 8.34 Hz

结论：local_jump 不是简单换成 local_jump IF-Net checkpoint 或继续训练 ICCD 层就能解决。它的误差主要来自跳变位置和跳变前后 IF 段的结构没有被第二阶段显式建模。当前的 ICCD refinement head 更擅长连续、小幅、平滑修正，对突变点附近的非平滑 IF 变化表达能力不足。

5. 多专家候选混合实验

本轮尝试了多 IF-Net 专家候选输入，包括 balanced、polynomial、sinusoidal、local_jump 等多个第一阶段 checkpoint。

5.1 全局平均式候选混合

结果较差：

- validation 重构 SNR 约 13.45 dB
- validation IF MAE 约 14.40 Hz

主要原因：不同专家输出的 IF 曲线不一定是同一个物理分量的同一个候选。直接做全局可学习平均时，可能把几条合理曲线平均成一条不真实的中间曲线，尤其在 crossing、local_jump、相切或短时重合场景中更明显。

5.2 基于 ICCD 残差的动态候选混合

动态混合后，模型会根据每个候选 IF 的初步 ICCD 重构残差来给权重。

结果仍不适合作为主路径：

- aggregate 重构 SNR 约 16.54 dB
- aggregate IF MAE 约 11.45 Hz
- crossing 出现明显退化，重构 SNR 约 4.32 dB, IF MAE 约 30.43 Hz

原因：ICCD 残差本身并不总能区分“物理身份正确的 IF”和“短时间内也能解释能量但身份错误的 IF”。在 crossing 场景中，候选曲线可能局部重构误差不大，但分量身份已经交换，导致后续 ICCD 展开层沿错误轨迹分解。

结论：多专家不是不能用，但不能只靠残差选择。后续如果使用多专家，需要加入场景判别器或身份一致性约束，让路由器先判断信号结构，再决定使用哪个专家或怎样融合 top-k 候选。

6. 当前最佳使用建议

当前第二阶段最稳组合：

- 通用模型：stage2_iccd/runs/all_from_separated/latest.pt
- sinusoidal_fm 专门模型：stage2_iccd/runs/sinusoidal_frozen/latest.pt

建议：

1. 一般场景优先使用 all_from_separated。
2. 如果前端判别器较确定是 sinusoidal_fm，可以切换到 sinusoidal_frozen。
3. 暂时不要把 all_multiexpert_dynamic 作为主模型，因为它在 crossing 上不稳定。
4. local_jump 暂时继续使用通用模型，直到第二阶段加入显式跳变位置建模。

7. 对第二阶段任务的影响

第二阶段的核心目标是用可微 ICCD 展开层把第一阶段的 IF 初始估计转化为可重构、可反传、可联合优化的分量分解过程。

从当前结果看：

- 简单场景和多数连续 IF 场景已经能支撑第二阶段继续推进。
- crossing 当前通用模型表现可接受，但多专家路由会引入身份交换风险，因此后续必须保留 top-2 候选和身份一致性检查。
- local_jump 会影响第二阶段的端到端精修，因为跳变点附近 IF 误差会直接导致相位积分误差，进而影响 ICCD 字典构造和分量重构。
- sinusoidal_fm 需要专门路由或更强的周期性 IF refinement，但问题比 local_jump 更容易通过专门模型缓解。

8. 下一步优化方向

优先级建议如下：

1. 为 local_jump 增加跳变位置辅助输入或辅助头。
 - 第一阶段已经有跳变事件定位思路，但第二阶段目前没有显式使用它。
 - 应把 jump position、jump confidence 或 jump mask 输入到 refinement head。
 - refinement head 在跳变点附近允许更大、更局部的 IF 修正。
2. 把 ICCD 展开层改成局部/分段式 IF refinement。
 - 对 local_jump，不能只用全局平滑修正。
 - 可以在跳变点前后分别建模 IF 曲线或分别设置正则强度。
3. 为 crossing 加身份一致性约束。
 - 保留 top-2 候选。
 - 在时间轴上限制分量身份频繁交换。
 - 训练损失中加入轨迹连续性或最优匹配约束。
4. 重新设计多专家路由器。
 - 不建议只用 ICCD 残差。

- 应结合信号类型判别器、候选置信度、身份一致性和局部重构误差。

5. 在通用模型稳定后，再逐步解冻 IF-Net。

- 目前仍建议先固定第一阶段 IF-Net，只训练 alpha、候选权重和 refinement head。
- 等 local_jump 和 crossing 的第二阶段机制补足后，再做端到端联合训练。

9. 当前是否可以继续第二阶段

可以继续第二阶段，但需要分层推进：

- 对 linear、quadratic、cubic、near_parallel、tangent_or_overlap、crossing：可以继续使用当前通用模型推进可微 ICCD 展开层和端到端训练框架。
- 对 sinusoidal_fm：可以使用专门模型作为临时分支，并继续优化路由。
- 对 local_jump：不建议直接进入大规模联合训练，应先补充跳变位置辅助头或分段式 refinement，否则端到端训练可能把误差传回 IF-Net，造成不稳定的错误修正。

因此，当前判断是：第二阶段框架已经成立，通用模型基本可用；但 local_jump 需要作为下一轮重点结构优化，而不是只依赖更长训练。

10. 2026-07-08 补充：先打牢简单多分量

根据“先专攻单分量与简单多分量，交叉、跳变等困难类型后续再攻克”的路线，本轮先没有把 crossing、local_jump 等困难类型继续混入训练，而是集中处理 linear、quadratic、cubic 这三类简单分离多分量信号。

10.1 为什么暂时没有直接做真正单分量

当前第二阶段使用的冻结 IF-Net checkpoint 是按 2 分量输出训练的。如果直接把真实 1 分量样本喂给第二阶段，第二条不存在的分量会带来两个问题：

- component loss 可以用零分量匹配处理，但 IF loss 仍会惩罚第二条“并不存在”的 IF；
- 可微 ICCD 层会尝试解释不存在的第二分量，可能把噪声当作弱分量重构。

因此，真正单分量需要先加入 inactive component mask 或 active-component loss mask。为了保证优化路径干净，本轮先做“简单、分离、2 分量”。

10.2 simple_multicomponent_long

配置：configs/simple_multicomponent_long.yaml

训练路径：

- 从 stage2_iccd/runs/separated_frozen/latest.pt 续训；
- 只使用 linear、quadratic、cubic；
- 训练噪声先限制为 white + colored；
- SNR 范围设为 4 dB 到 28 dB；
- 继续训练 800 步。

训练后 checkpoint：stage2_iccd/runs/simple_multicomponent_long/latest.pt

与原 separated_frozen 在同一 easy 条件下比较：

模型	aggregate SNR / dB	aggregate IF MAE / Hz	smooth
separated_frozen	25.62	2.20	9.21
simple_multicomponent_long	25.93	1.64	2.59

分场景结果：

场景	SNR / dB	IF MAE / Hz
linear	26.74	1.30
quadratic	25.94	1.58
cubic	25.12	2.06
aggregate	25.93	1.64

结论：这一步达到了较好的效果。IF 精度明显提升，曲线二阶平滑度明显降低，说明第二阶段 refinement head 没有只是追求重构误差，而是把 IF 轨迹也修得更稳。

10.3 鲁棒条件复测

在更复杂条件下评估：

- SNR 范围：-2 dB 到 24 dB；
- 噪声：white 0.55、colored 0.25、impulsive 0.10、trend 0.10。

模型	aggregate SNR / dB	aggregate IF MAE / Hz	smooth
separated_frozen	22.94	2.91	13.68
simple_multicomponent_long	22.86	2.55	4.46

结论：鲁棒条件下重构 SNR 基本持平，IF MAE 和 smoothness 仍然更好。因此 simple_multicomponent_long 可以作为当前简单多分量阶段的最佳模型。

10.4 simple_multicomponent_robust 尝试

配置：configs/simple_multicomponent_robust.yaml

训练路径：

- 从 simple_multicomponent_long/latest.pt 续训；
- 训练时加入 impulsive 和 trend 噪声；
- 继续训练 600 步。

结果：

评估条件	SNR / dB	IF MAE / Hz	smooth
easy	25.53	1.74	2.21
robust	22.48	2.58	3.23

结论：鲁棒续训没有超过 simple_multicomponent_long。它让 IF 曲线更平滑，但重构 SNR 略降，且 robust IF MAE 没有实质改善。因此它目前只作为诊断实验保留，不作为第一步最佳 checkpoint。

10.5 当前第一步判断

当前简单多分量阶段可以视为初步达标：

- easy 条件 aggregate IF MAE 已降到约 1.64 Hz；
- robust 条件 aggregate IF MAE 约 2.55 Hz；
- linear/quadratic/cubic 都没有明显失控；
- 可微 ICCD 层的 alpha、候选权重和 IF refinement head 都能稳定训练。

下一步不建议立刻攻 local_jump。更稳的顺序是：

1. 增加 active-component mask，补真正单分量训练和评估；
2. 在简单多分量基础上加入 near_parallel 或更小间隔的非交叉多分量；
3. 简单和近邻场景都稳定后，再进入 crossing；
4. 最后单独攻 local_jump，并加入跳变位置辅助头。

11. 2026-07-08 补充：单分量与单/多分量合并尝试

11.1 active-component mask

为了正确处理真正单分量，代码中新增了 active-component mask：

- 仿真器现在可以通过 active_components 指定真实活跃分量数；
- inactive component 的幅值设为 0；
- component/reconstruction loss 仍约束模型不要重构不存在的分量；
- IF loss 只在 active component 上计算，避免用不存在的第二条 IF 污染训练。

这一步是必要的。否则单分量训练会把“第二条不存在的 IF”当作真实目标，造成错误优化。

11.2 单分量专用模型

配置：configs/simple_single_component.yaml

训练路径：

- 从 simple_multicomponent_long/latest.pt 续训；
- 只使用 linear、quadratic、cubic；
- active_components: 1；
- 训练 500 步。

checkpoint: stage2_iccd/runs/simple_single_component/latest.pt

单分量 easy 条件评估：

场景	SNR / dB	IF MAE / Hz	component L1
linear	26.64	0.65	0.029
quadratic	27.71	0.69	0.025
cubic	26.83	0.73	0.028
aggregate	27.06	0.69	0.028

单分量 robust 条件评估：

场景	SNR / dB	IF MAE / Hz	component L1
linear	23.35	0.73	0.041
quadratic	23.87	0.78	0.039
cubic	23.71	0.82	0.040
aggregate	23.64	0.78	0.040

结论：单分量专用模型效果很好，说明 active-component mask 是有效的。

11.3 单分量模型不能直接处理双分量

把 simple_single_component/latest.pt 放到 2 active components 的 easy 条件下评估：

条件	SNR / dB	IF MAE / Hz	component L1
2 分量 easy	7.57	7.07	0.223

结论：单分量模型非常专用，不能直接用于双分量。它会倾向于只解释一个主分量，导致另一个真实分量重构失败。

11.4 双分量模型处理单分量的表现

把 simple_multicomponent_long/latest.pt 放到 1 active component 的 easy 条件下评估：

条件	SNR / dB	IF MAE / Hz	component L1
1 分量 easy	28.49	1.32	0.189

结论：双分量模型对单分量的重构 SNR 和 IF 还可以，但 component L1 很高，说明它会把单个真实分量拆到两个输出槽位中。这对后续可解释分量分解是不利的。

11.5 mixed-active 统一模型尝试

配置：configs/simple_active_mixed.yaml

训练路径：

- 从 simple_multicomponent_long/latest.pt 续训；
- 训练样本中 45% 为单分量，55% 为双分量；
- 训练 700 步。

评估结果：

条件	SNR / dB	IF MAE / Hz	component L1
1 分量 easy	27.58	1.53	0.063
2 分量 easy	23.71	2.96	0.061
mixed robust	22.71	2.70	0.070

结论：mixed-active 统一模型没有超过专用模型组合。它比单分量专用模型差，也比双分量专用模型差，说明单/多分量在当前第二阶段结构中存在明显任务冲突。

11.6 当前最稳组合

当前不建议强行用一个模型同时处理单分量和双分量。最稳组合是：

- 单分量: stage2_iccd/runs/simple_single_component/latest.pt
- 简单双分量: stage2_iccd/runs/simple_multicomponent_long/latest.pt

进入下一步之前, 建议先加 active-component 判别器或能量型路由器:

- 若判定为 1 active component, 走单分量专用模型;
- 若判定为 2 active components, 走简单双分量模型;
- 如果判别置信度低, 保留 top-2 active-count 候选, 进入第二阶段时同时输出置信度。

因此, 当前简单单分量和简单双分量都已经有了可用模型, 但还需要一个轻量 active-count 路由器, 才能形成完整稳定的第一层处理流程。

12. 2026-07-08 补充: active-count 路由器与 near_parallel 纳入

在完成单分量专用模型和简单双分量专用模型之后, 本轮继续补上了一个轻量 active-count 路由器。它的任务不是直接预测完整 IF, 而是先判断输入信号更像 1 个活跃分量还是 2 个活跃分量, 然后把样本送到对应的第二阶段模型:

- 1 active component: 使用 stage2_iccd/runs/simple_single_component/latest.pt;
- 2 active components: 使用 stage2_iccd/runs/simple_multicomponent_long/latest.pt。

这样做的原因是, 前面的实验已经说明 “一个统一模型同时处理单分量和双分量” 会出现明显任务冲突。单分量模型很擅长单分量, 但不能直接处理双分量; 双分量模型能重构单分量, 却容易把一个真实分量拆到两个输出槽位里。因此先做 active-count 判别, 再走专门模型, 比强行用一个模型覆盖所有情况更稳。

12.1 新增代码

本轮新增了以下代码和配置:

- src/stage2_iccd/active_count.py: active-count 分类网络。输入来自多尺度 STFT 图和辅助统计特征, 输出 active_1 / active_2 两类概率、置信度和 margin;
- src/stage2_iccd/train_active_count.py: active-count 路由器训练脚本;
- src/stage2_iccd/eval_active_count.py: active-count 单独评估脚本;
- src/stage2_iccd/eval_active_routed_stage2.py: 把 active-count 路由器接到单分量/双分量 Stage2 模型后的端到端评估脚本;
- configs/active_count_simple.yaml: 只包含 linear、quadratic、cubic 的初始路由器配置;
- configs/active_count_simple_near_parallel.yaml: 在简单场景基础上加入 near_parallel 的路由器配置。

12.2 初始 active-count 路由器

先用 active_count_simple.yaml 训练, 只覆盖 linear、quadratic、cubic 的 1/2 active component 判别。

在简单 easy 条件下:

评估范围	accuracy	active_1 accuracy	active_2 accuracy
linear/quadratic/cubic	99.1%	99.7%	98.5%

在 robust 条件下:

评估范围	accuracy	active_1 accuracy	active_2 accuracy
linear/quadratic/cubic	97.3%	98.1%	96.5%

接到 Stage2 后，linear/quadratic/cubic 的 routed 结果为：

条件	route accuracy	IF MAE / Hz	重构 SNR / dB
easy	99.4%	1.30	26.01
robust	97.7%	1.85	23.09

结论：只看简单场景时，active-count 路由器已经足够稳定，可以把单分量和双分量模型组合起来使用。

12.3 near_parallel 的问题与修正

继续把同一个 active-count 路由器放到 near_parallel 上评估，发现准确率只有约 90%：

条件	accuracy	active_1 accuracy	active_2 accuracy
near_parallel easy	90.3%	85.6%	95.0%
near_parallel robust	90.2%	88.9%	91.6%

这个结果说明 near_parallel 的主要问题不在第二阶段 ICCD 重构本身，而在“路由器没有见过这类结构”。near_parallel 的时频图中，两条 IF 长时间接近并近似平行，能量分布容易被误判成一个宽一些的单分量脊线；反过来，某些单分量的缓慢弯曲也可能被误判成两个贴近分量。因此如果 active-count 路由器只用普通 separated linear/quadratic/cubic 训练，面对 near_parallel 会不稳定。

为了修正这个问题，本轮训练了 active_count_simple_near_parallel.yaml，把 near_parallel 一并加入 active-count 训练。

修正后，near_parallel 路由准确率明显提高：

条件	accuracy	active_1 accuracy	active_2 accuracy
near_parallel easy	99.1%	100.0%	98.3%
near_parallel robust	97.6%	99.8%	95.3%

同时重新检查 linear/quadratic/cubic，没有发生严重遗忘：

条件	accuracy	active_1 accuracy	active_2 accuracy
simple easy	98.6%	100.0%	97.2%
simple robust	96.8%	100.0%	93.7%

这里要注意一个小代价：加入 near_parallel 后，强噪声下简单双分量的路由准确率从约 96.5% 降到约 93.7%。这说明路由器容量和训练分布仍然有限，但端到端 Stage2 指标仍然可用。

12.4 新路由器接入 Stage2 后的结果

使用新路由器 active_count_simple_near_parallel/latest.pt，并接入：

- 单分量模型：simple_single_component/latest.pt；
- 双分量模型：simple_multicomponent_long/latest.pt。

在 linear、quadratic、cubic、near_parallel 四类上做 routed Stage2 评估，结果如下：

条件	route accuracy	IF MAE / Hz	重构 SNR / dB	component L1
easy	98.8%	1.18	26.21	0.039
robust	97.3%	1.64	23.42	0.050

分场景看，单分量 IF MAE 基本稳定在 0.62-0.86 Hz；双分量中 near_parallel 的 IF MAE 为 easy 1.30 Hz、robust 1.66 Hz，说明第二阶段 ICCD 对 near_parallel 本身并不弱。当前 near_parallel 的主要风险已经从“重构能力不足”转移到“路由器是否足够稳定”。

12.5 当前判断

到目前为止，第一批可继续推进的第二阶段场景包括：

- 单分量 linear / quadratic / cubic；
- 简单双分量 linear / quadratic / cubic；
- near_parallel。

这些场景已经满足继续推进的基本门槛：路由准确率在 easy/robust 条件下都接近或超过 97%，端到端 IF MAE 在 1-2 Hz 左右，重构 SNR 大约 23-26 dB。

暂时不建议直接把 crossing 和 local_jump 加进同一个训练任务。原因是：

- crossing 的主要风险是分量身份交换，不能只靠 active-count 路由解决；
- local_jump 的主要风险是跳变位置定位和跳变点附近的分段 IF 修正，需要显式 jump head 或 jump mask；
- sinusoidal_fm 比 crossing/local_jump 更适合作为下一步，因为它仍属于连续 IF，只是周期性调制更强，适合作为从简单连续场景到困难场景之间的过渡。

12.6 下一步优化顺序

下一轮建议按下面顺序推进：

1. 把 sinusoidal_fm 纳入当前 active-count + routed Stage2 流程，先看路由是否稳定，再看 Stage2 是否需要专门分支；
2. 如果 sinusoidal_fm 的路由稳定但 IF MAE 偏高，优先训练 sinusoidal_fm 的 Stage2 分支，而不是马上解冻 IF-Net；
3. 再进入 crossing，重点加入身份一致性约束和 top-2 候选保留；
4. 最后集中处理 local_jump，加入跳变位置辅助头、jump mask 或分段 refinement。

因此，本轮结论是：简单单/双分量和 near_parallel 的 Stage2 路由框架已经基本打通，可以继续往 sinusoidal_fm 扩展；但 crossing/local_jump 仍应作为后续专项攻克对象。

13. 2026-07-09 补充：多项式双分量尾部误差专项优化

在重新生成“旧模型 vs 新模型”的可视化图之后，可以看到当前第二阶段已经能让多数简单单分量、简单双分量和 near_parallel 的曲线更接近真实 IF。但有一个比较明显的弱点：quadratic active=2 这类双分量多项式信号仍然存在尾部误差。也就是说，平均效果已经不错，但某些样本的局部弯曲、端点区域或两分量接近区域会出现偏离。

这类问题和 crossing/local_jump 不完全一样。crossing 的核心问题是身份交换，local_jump 的核心问题是跳变点定位；而 quadratic/cubic active=2 的主要问题是连续曲线的弯曲形状和双分量间距同时变化，第二阶段如果只按普通 separated 双分量训练，容易学到比较平滑、保守的修正，无法完全覆盖多项式尾部样本。

13.1 本轮新增的诊断工具

本轮新增了一个样本级 checkpoint 对比脚本：

- `stage2_iccd/scripts/compare_stage2_checkpoints.py`

它的作用是让两个 Stage2 checkpoint 在同一批仿真样本上逐个比较，而不是只看一次汇总平均值。脚本会输出：

- 两个 checkpoint 的 IF MAE 均值、中位数、p90、p95；
- 重构 SNR 均值；
- 新 checkpoint 相比旧 checkpoint 的逐样本胜率；
- `comparison.json` 和 `comparison.csv`，方便后续定位“哪些样本被改善，哪些样本被恶化”。

这个脚本很重要，因为当前任务不能只看平均值。如果某个模型平均值略好，但 p90/p95 或其他场景明显变差，它就不适合作为默认模型。

13.2 多项式专项双分量模型

首先训练了 `poly_multicomponent_refine`，配置文件为：

- `stage2_iccd/configs/poly_multicomponent_refine.yaml`

这个模型从当前简单双分量稳定 checkpoint 继续训练：

- 初始化 checkpoint: `stage2_iccd/runs/simple_multicomponent_long/latest.pt`；
- 固定为 2 active components；
- 加大 quadratic 和 cubic 的采样权重；
- 保留少量 linear 和 near_parallel，避免模型完全只记住多项式样本；
- 适当放宽 IF refinement 幅度，让模型能修正更大的局部偏差。

训练后的分场景评估如下。

条件	linear IF MAE / Hz	quadratic IF MAE / Hz	cubic IF MAE / Hz	near_parallel IF MAE / Hz	平均 IF MAE / Hz	SNR / dB
easy	1.01	1.70	2.10	1.03	1.46	25.76
robust	3.99	2.47	3.50	1.36	2.83	22.58

从这个表可以看出，`poly_multicomponent_refine` 对多项式双分量确实有帮助，但它不是一个适合直接替换默认模型的结果。原因是 robust 条件下 linear 明显退化，IF MAE 到了约 3.99 Hz。这说明它学到了更偏向弯曲多项式的修正策略，对线性双分量不够稳。

为了更公平地判断它是否真的改善 quadratic active=2，又做了 160 个样本的逐样本对比。对比对象是当前默认双分量模型 `simple_multicomponent_long`。

条件	默认模型 mean / Hz	多项式专项 mean / Hz	默认模型 p95 / Hz	多项式专项 p95 / Hz	专项模型胜率
quadratic easy	1.60	1.57	4.89	4.72	60.0%
quadratic robust	2.37	2.32	8.84	8.65	65.6%

这个结果说明：多项式专项模型确实能略微降低 quadratic active=2 的平均误差和 p95 误差，而且超过一半样本会变好。但改善幅度还不够大，p90 在 robust 条件下甚至有轻微变差。因此它目前更适合作为“诊断模型”或后续 hard-sample mining 的起点，而不是默认模型。

13.3 均衡双分量微调尝试

为了避免多项式专项模型牺牲 linear，本轮又训练了一个更均衡的模型：

- stage2_iccd/configs/balanced_multicomponent_refine.yaml

它仍然从 simple_multicomponent_long/latest.pt 继续训练，但不再过度偏向 quadratic/cubic，而是在 linear、quadratic、cubic、near_parallel 之间做相对温和的采样平衡。

评估结果如下。

条件	linear IF MAE / Hz	quadratic IF MAE / Hz	cubic IF MAE / Hz	near_parallel IF MAE / Hz	平均 IF MAE / Hz	SNR / dB
easy	1.44	1.87	1.89	1.15	1.59	25.82
robust	2.85	3.22	3.13	1.80	2.75	22.58

这个结果没有超过当前默认双分量模型。它对 easy 条件下的 cubic 有一点帮助，但 linear、quadratic、near_parallel 和 robust 条件整体变差。因此 balanced_multicomponent_refine 也不应作为默认模型。

13.4 当前采用的模型选择

本轮优化后的结论是：目前最稳的默认组合仍然保持不变。

- 单分量: stage2_iccd/runs/simple_single_component/latest.pt
- 简单双分量与 near_parallel: stage2_iccd/runs/simple_multicomponent_long/latest.pt
- active-count 路由器: stage2_iccd/runs/active_count_simple_near_parallel/latest.pt

新增的两个模型不删除，但暂时作为诊断和后续研究材料保留：

- stage2_iccd/runs/poly_multicomponent_refine/latest.pt: 多项式双分量专项模型，能轻微改善 quadratic 尾部，但会损害 robust linear;
- stage2_iccd/runs/balanced_multicomponent_refine/latest.pt: 均衡微调模型，没有超过默认双分量模型。

13.5 为什么继续盲目训练不一定有效

这轮实验说明，当前问题不只是“训练不够久”。如果简单延长训练或提高某一类样本权重，模型会在某些样本上改善，但容易把别的场景拉坏。根本原因可能有三点：

1. Stage2 的 IF refinement head 仍然比较轻量，面对二次/三次多项式的局部曲率变化时，表达能力有限；
2. Stage1 提供的 top-k 候选如果在困难样本里已经偏离真实 IF，Stage2 只能在有限范围内修正，不能凭空恢复完全正确的分量形状；
3. 当前 active-count 路由器只能判断 1/2 active components，不能区分 linear、quadratic、cubic 这样的多项式子类型，因此不能安全地把多项式专项模型只分配给真正需要它的样本。

因此，下一步更合理的优化方向不是直接替换默认模型，而是做“质量感知”的分支选择：

- 给 Stage2 增加候选置信度或重构残差门控；
- 对 quadratic/cubic active=2 的高误差样本做 hard-sample mining；
- 训练一个轻量 subtype / quality gate，用来判断样本是否需要走多项式专项分支；
- 继续保留当前默认双分量模型作为基线，任何新模型必须同时比较 mean、p90、p95 和跨场景退化情况。

13.6 对进入下一步的影响

这轮结果不会阻止进入后续第二阶段工作。原因是第二阶段的目标不是让初始 IF 完全贴合真实脊线，而是提供一个足够可靠的初始估计，让可微 ICCD 展开层能够稳定重构并反向微调。当前单分量、简单双分量和 near_parallel 已经满足这个要求。

但对于困难多分量多项式样本，需要注意：

- 如果只是做重构，当前默认模型已经基本可用；
- 如果要求 IF 曲线在图上非常贴近真实曲线，quadratic/cubic active=2 的尾部样本仍需要继续优化；
- 如果后续进入 crossing/local_jump，不能把这类误差简单归因于 ICCD 层，而要同时检查 Stage1 候选质量、身份一致性、跳变位置辅助头和路由稳定性。

所以当前阶段的建议是：默认流程继续使用稳定组合；多项式专项模型作为备选诊断分支保留；下一步优先做候选质量/置信度门控和 hard-sample mining，再继续扩展到 sinusoidal_fm、crossing 和 local_jump。

14. 2026-07-09 补充：默认双分量与多项式专项分支的质量门控

上一节说明，多项式专项模型不能直接替换默认双分量模型，因为它在某些样本上改善 IF，但也可能让其他样本变差。为了继续优化，本轮没有继续盲目加训练轮数，而是做了一个更接近实际部署的问题：如果系统同时保留默认双分量分支和多项式专项分支，能不能根据模型自己的输出质量，自动选择更可信的一支？

14.1 新增脚本

本轮新增两个诊断脚本：

- stage2_iccd/scripts/evaluate_stage2_quality_gate.py
- stage2_iccd/scripts/sweep_stage2_quality_gate.py

第一个脚本会在同一批样本上同时运行：

- 默认双分量模型：simple_multicomponent_long/latest.pt；
- 多项式专项模型：poly_multicomponent_refine/latest.pt。

然后它记录四种结果：

- default：永远使用默认双分量模型；
- specialist：永远使用多项式专项模型；
- gated：根据无监督质量分数在两个分支中选择；
- oracle：根据真实 IF 误差选择更好分支，只作为理论上限，实际部署不能使用。

无监督质量分数主要来自“重构后和观测信号的残差”，并加入 IF 修正量和平滑度惩罚。它的意义是：真实部署时拿不到真实 IF，但可以看到当前分支重构出来的信号是否贴近输入信号，以及 IF 修正是否过大、是否过度抖动。

第二个脚本会在保存下来的 CSV 上扫描不同的 penalty 和 margin，不重新跑模型，用来判断这种简单无监督门控的上限。

14.2 初始保守门控的结果

先使用比较保守的设置：

- `delta_penalty=0.015`
- `smooth_penalty=0.000002`
- `score_margin=0.05`

这个设置要求专项分支的质量分数明显高于默认分支，才切换到专项分支。

在 easy 条件下，结果为：

选择方式	IF MAE mean / Hz	IF MAE p95 / Hz	使用专项分支比例	oracle 匹配率
default	1.617	3.783	0.0%	38.1%
specialist	1.566	3.861	100.0%	61.9%
gated	1.611	3.800	18.1%	39.4%
oracle	1.515	3.783	61.9%	100.0%

在 robust 条件下，结果为：

选择方式	IF MAE mean / Hz	IF MAE p95 / Hz	使用专项分支比例	oracle 匹配率
default	2.236	8.283	0.0%	28.1%
specialist	2.141	8.139	100.0%	71.9%
gated	2.225	8.283	13.1%	32.5%
oracle	2.109	8.139	71.9%	100.0%

这个结果说明，保守门控太保守了。它只在 13%-18% 的样本上使用专项分支，而 oracle 显示实际上约 62%-72% 的样本使用专项分支会更好。因此，仅靠“专项分支必须明显更好”这个规则，会错过大部分可改善样本。

14.3 离线 sweep 后的较优门控

继续对质量分数参数做离线 sweep，合并 easy 和 robust 共 640 个样本，得到一个更合理的门控倾向：

- `delta_penalty=0.03`
- `smooth_penalty=0.0`
- `margin=-0.04`

这个设置的含义是：默认双分量模型作为兜底分支，多项式专项分支作为更积极的候选；只要专项分支没有明显比默认分支差，就优先使用专项分支。

合并 easy + robust 的结果如下。

选择方式	IF MAE mean / Hz	IF MAE p90 / Hz	IF MAE p95 / Hz	使用专项分支比例	oracle 匹配率
default	1.927	2.516	6.695	0.0%	33.1%
specialist	1.853	2.358	6.623	100.0%	66.9%
gated sweep-best	1.847	2.322	6.615	89.7%	70.0%
oracle	1.812	2.322	6.615	66.9%	100.0%

这个结果比保守门控更有价值。它说明“默认 + 专项 + 质量门控”确实可以比单独使用默认模型更好，也略好于永远使用专项模型。但提升幅度仍然不大，说明当前的无监督质量特征还比较粗糙。

14.4 当前结论

质量门控给出了三个重要判断：

1. 多项式专项分支不是无用分支。很多样本上它比默认双分量模型更好，尤其在 robust 条件下，oracle 使用专项分支的比例达到约 72%。
2. 单纯看重构残差不够可靠。重构更贴近观测信号，不一定代表 IF 曲线更贴近真实 IF，因为噪声、分量交换和局部过拟合都会影响残差。
3. 当前最实用的门控策略不是“只有专项明显更好才切换”，而是“专项优先，默认兜底”。这说明多项式专项训练方向是有用的，但还需要更强的质量判别特征。

因此，下一步不建议直接把 `poly_multicomponent_refine` 替换为默认双分量模型，而是建议继续做一个真正的 supervised quality head。这个 quality head 可以用仿真数据训练，输入包括：

- 两个分支的观测重构残差；
- IF refinement 的修正幅度；
- IF 曲线平滑度和局部曲率；
- 两个候选 IF 之间的差异；
- Stage1 候选置信度和 top-2 间距；
- active-count 路由器置信度。

训练目标不是预测真实场景类型，而是预测“哪个分支的 IF 误差更小”或“当前样本是否属于高风险尾部样本”。这样比手工门控更贴近最终任务。

14.5 对后续路线的影响

当前默认流程仍保持：

- active-count 路由器先判断 1/2 active components；
- 单分量走 `simple_single_component`；
- 简单双分量和 `near_parallel` 走 `simple_multicomponent_long`；
- 多项式专项分支暂时不作为默认替换，而作为候选分支和 hard-sample mining 工具。

但从这一轮开始，后续优化方向已经从“继续训练一个更大的统一模型”转为“带质量感知的多分支选择”。这和后续处理 crossing/local_jump 的思路是一致的：困难样本不一定靠一个模型硬吃掉，而是先保留多个候选，再用置信度、身份一致性、跳变位置或重构质量来决定如何进入可微 ICCD 展开层。

因此，下一步建议优先实现 supervised quality head，并把它接到 routed Stage2 评估中。只有当 quality head 能稳定超过手工门控和永远使用专项分支，才把多项式专项分支纳入默认推理流程。

15. 2026-07-09 补充：按 P0 优先级落地的第一轮优化

本轮按照“先补关键信息通道和质量判断，再继续大改结构”的原则推进。由于当前 Stage2 的主干已经能稳定做重构，改动重点没有放在重新训练一个更大的网络，而是放在四个更靠近问题根源的位置：

1. 用监督式质量选择头替代纯手工门控；
2. 在 component loss 中显式处理 active / inactive 分量，减少单分量泄漏；
3. 给 Stage2 refinement head 预留 jump_mask 或 jump_prob 条件输入；
4. 在质量选择头训练中加入类别均衡和难例权重，避免选择器塌缩成“永远选某一个分支”。

15.1 监督式 Stage2 质量选择头

新增代码：

- stage2_iccd/src/stage2_iccd/quality_selector.py
- stage2_iccd/scripts/train_stage2_quality_selector.py
- stage2_iccd/scripts/eval_stage2_quality_selector.py

这个质量选择头的目标不是判断信号类型，而是判断“默认双分量分支”和“多项式专项分支”哪一个在当前样本上更可靠。训练标签由仿真数据自动生成：同一个样本同时经过两个 Stage2 分支，然后计算它们各自相对真实 IF 的 MAE，误差更小的分支作为监督标签。

输入特征没有使用真实 IF，因此未来推理时也能获得。当前使用的特征包括：

- 默认分支和专项分支的重构残差；
- 两个分支重构残差的差值与比例；
- 两个分支 refined IF 的差异统计量；
- IF 曲线的范围、斜率、平滑度和曲率；
- 候选权重熵，用来粗略表示候选 IF 的不确定性；
- 两个分支输出之间的整体偏离程度。

第一版直接训练时出现了一个问题：选择器倾向于永远选择多项式专项分支。独立评估结果如下：

选择方式	IF MAE mean / Hz	IF MAE p90 / Hz	IF MAE p95 / Hz	使用专项分支比例
default	2.294	3.997	9.296	0.0%
specialist	2.218	3.907	9.288	100.0%
selected	2.218	3.907	9.288	100.0%
oracle	2.175	3.830	9.158	66.3%

这说明监督标签本身不是问题，问题在于标签分布和分支差异都偏向专项分支，普通交叉熵很容易学成“只选专项”。因此我又加入了两项修正：

- 类别均衡：在一个 batch 内自动提高少数类标签的权重；
- margin 加权：两个分支 MAE 差距越大的样本权重越高，差距很小的样本权重降低。

平衡训练后的 best.pt 独立评估如下：

选择方式	IF MAE mean / Hz	IF MAE p90 / Hz	IF MAE p95 / Hz	使用专项分支比例	选择准确率
default	2.294	3.997	9.296	0.0%	-
specialist	2.218	3.907	9.288	100.0%	-
selected	2.221	3.907	9.288	79.6%	65.3%
oracle	2.175	3.830	9.158	66.3%	100.0%

这个结果比第一版健康，因为它不再永远选择同一个分支，并且 near_parallel 场景下 selected 的均值略好于 specialist。但从整体均值看，它仍然没有稳定超过“永远使用专项分支”。因此当前结论是：监督式 quality head 的代码路径已经打通，可以作为诊断和后续融合基础；但它暂时不应该替代默认推理策略。

15.2 active-component loss 与单分量泄漏

原来的 Stage2 已经在 IF loss 中使用了 active mask，也就是说真实不存在的分量不会强迫 IF 贴近某条伪曲线。但 component reconstruction loss 之前没有充分利用 active mask，这会带来一个隐患：当输入只有一个真实分量时，双分量模型可能把这个真实分量拆到两个输出槽里，看起来总重构还不错，但分量解释是错的。

本轮在 stage2_iccd/src/stage2_iccd/losses.py 中新增了：

- active_component_permutation_mse
- active_component_permutation_l1

它们先根据 active mask 做分量匹配，只对真实活跃分量计算主要匹配误差；同时对未匹配的 inactive 输出槽增加能量惩罚。这样模型不能再通过“多生成一条弱分量”来逃避单分量约束。

短训练探针已经验证新增指标可以正常进入训练日志：

指标	探针结果
if_mae_hz	1.109 Hz
rec_snr_db	28.48 dB
component_mse	0.0619
inactive_component_mse	0.0546

这只是 smoke 级验证，说明代码路径正常，不等价于已经把单分量泄漏彻底压到目标值以下。后续需要做更长训练，并单独统计 single active 输入下的 inactive component energy。

15.3 jump 条件输入接口

针对 local_jump，本轮先没有直接重训一个大模型，而是在 Stage2ICCDModel 和 IFRefinementHead 中加入了条件输入接口：

- Stage2ModelConfig.refine_extra_channels
- Stage2ICCDModel.forward(..., refinement_extra=...)
- IFRefinementHead.forward(..., extra=...)

这样后续可以把 Stage1 的 jump_mask、jump_prob 或 jump_center 辅助输出拼到 refinement head 的输入里。默认配置仍然是 refine_extra_channels=0，因此旧 checkpoint 不受影响。

这一点目前属于“结构预留”，还不是完整 local_jump 优化。真正让它产生收益，还需要两步：

1. 在 Stage1 输出中稳定导出每个候选 IF 对应的 jump probability 或 jump center；
2. 训练 Stage2 local_jump 专项模型，让 refinement head 学会在 jump 区域放宽平滑约束，在非 jump 区域保持强正则。

15.4 当前验证结果与是否进入下一步

本轮代码级验证均通过：

- compileall 通过；
- stage2_iccd.smoke_test 通过，ICCD 的 alpha、候选权重和 refinement head 梯度正常；
- simple_active_mixed 短训练通过，并产生 inactive_component_mse；
- supervised quality selector 能训练、保存、加载和独立评估。

但从任务效果看，还不能把 supervised quality selector 设为默认策略。原因很清楚：它虽然避免了完全塌缩，但整体 IF MAE 还没有稳定超过永远使用多项式专项分支。也就是说，P0 里的“监督式质量选择头”已经完成工程闭环，但还没有达到“默认部署”的效果闭环。

下一步更合理的优化顺序是：

1. 先扩大 quality selector 的特征，加入 Stage1 top-2 置信度、active-count 置信度、分支间身份一致性和局部曲率突变特征；
2. 对 active-component loss 做更长训练，单独评估单分量输入下的 inactive component energy；
3. 接入 Stage1 的 jump auxiliary 输出，开始训练 jump_mask 条件化的 local_jump refinement；
4. 等 quality selector 能稳定超过 specialist 或至少稳定降低 p95，再把它接入默认 routed Stage2；
5. 最后再推进相位通道或多专家软融合，因为这两项会牵动 Stage1/Stage2 输入结构，改动范围更大。

16. 2026-07-09 补充：P0 后半段完成情况

本轮继续完成上一节列出的两个 P0 后半段任务：

1. 扩展 supervised quality selector 的输入特征，并重新训练；
2. 对 active-component loss 做更长训练，单独评估 single active 输入下 inactive component energy 是否真正下降。

16.1 quality selector 特征扩展

上一版 quality selector 主要依赖 Stage2 两个分支自己的输出，例如重构残差、IF 平滑度、候选权重熵等。这些特征能描述“当前分支看起来是否平滑、是否重构得好”，但缺少上游路由和分量数量信息。因此本轮新增了 stage2_iccd/src/stage2_iccd/quality_context.py，把两个额外上下文接入 quality selector：

- Stage1 hard router：读取 ifnet_stage1/runs/router_hard_v3/latest.pt，输出 top-1 置信度、top-2 margin，以及 poly / sinusoidal / cross / jump 四类概率；
- active-count router：读取 stage2_iccd/runs/active_count_simple_near_parallel/latest.pt，输出 active-count 置信度、margin 和 two-component 概率。

同时，在 stage2_iccd/src/stage2_iccd/quality_selector.py 中新增了两类几何特征：

- 身份一致性特征：两条 refined IF 的交叉翻转率、最小分量间隔，以及两个分支之间的差值；
- 局部曲率突变特征：二阶差分最大值与平均值的比值，用来提示局部突变或局部异常弯折。

扩展后，quality selector 的特征从原来的 24 维增加到 42 维。为了兼容旧 checkpoint，代码没有强制所有模型都使用 42 维，而是把 feature_names 存进 checkpoint。评估时会按 checkpoint 记录的特征名重新构造输入，因此旧的 24 维 selector 仍能评估。

16.2 扩展特征后的质量选择结果

训练命令使用：

```
.\venv_ifnet\Scripts\python.exe stage2_iccd\scripts\train_stage2_quality_selector.py --run-dir stage2_iccd/runs/stage2_quality_selector_p0_context --steps 800 --batch-size 8 --val-batches 28 --hidden 96 --dropout 0.10 --balance-classes --margin-scale-hz 0.65
```

独立评估 best.pt 的结果如下：

选择方式	IF MAE mean / Hz	IF MAE p90 / Hz	IF MAE p95 / Hz	使用专项分支比例	选择准确率
default	2.285	4.106	9.135	0.0%	-
specialist	2.211	3.936	9.071	100.0%	-
selected best	2.210	3.936	9.071	99.7%	66.7%
oracle	2.167	3.936	9.063	66.3%	100.0%

独立评估 latest.pt 的结果如下：

选择方式	IF MAE mean / Hz	IF MAE p90 / Hz	IF MAE p95 / Hz	使用专项分支比例	选择准确率
default	2.285	4.106	9.135	0.0%	-
specialist	2.211	3.936	9.071	100.0%	-
selected latest	2.226	3.999	9.093	58.1%	62.6%
oracle	2.167	3.936	9.063	66.3%	100.0%

这个结果说明，扩展特征确实让模型具备了更丰富的判断依据，但当前质量选择头仍然没有形成足够强的默认替代能力。latest.pt 能避免塌缩，但均值比 specialist 差；best.pt 均值略微好于 specialist，但几乎总是选择 specialist，说明它更像一个“专项优先、默认极少兜底”的策略，而不是真正稳定的二分支质量判别器。

因此，P0 的质量头部分目前可以认为“工程闭环完成”，但“不进入默认推理”。它的价值是：

- 代码路径已经支持 Stage1 top-2 置信度、active-count 置信度、身份一致性和局部曲率突变；
- 可以作为后续多专家软融合或更强质量判别器的基础；
- 暂时不能作为替代 simple_multicomponent_long 或 poly_multicomponent_refine 的默认决策器。

16.3 active-component loss 的长训与单分量泄漏

本轮先直接用 simple_active_mixed.yaml 训练了一个较强版本：

- run dir: stage2_iccd/runs/simple_active_mixed_active_loss_p0
- 训练后 single active 的 inactive_component_mse 从基线 0.0792 降到 0.0146
- 但 two active 的 IF MAE 从基线 1.537 Hz 退化到 2.659 Hz

这个版本说明 active loss 方向有效，但惩罚太强会伤害正常双分量。因此又新增了一个更保守的配置：

- stage2_iccd/configs/simple_active_mixed_p0_conservative.yaml

保守版做了几件事：

- single active 采样比例从 45% 降到 35%，保留更多双分量样本；
- inactive_component 权重降到 0.04；
- max_refine_hz 从 24 Hz 降到 18 Hz；
- smooth 和 delta 正则增强，避免 refinement head 过度弯折。

独立评估结果如下。

模型	active=1 inactive MSE	active=1 IF MAE / Hz	active=2 IF MAE / Hz	active=2 SNR / dB
simple_multicomponent_long 基线	0.0792	1.376	1.537	25.74
active_loss 强版本	0.0146	1.298	2.659	23.72
active_loss 保守版	0.0288	0.770	1.912	25.19

保守版的结论更合理：它没有把 inactive energy 压到最低，但相比基线仍下降约 64%；同时 single active IF MAE 明显改善，从 1.376 Hz 降到 0.770 Hz。代价是双分量 IF MAE 仍有一定退化，从 1.537 Hz 上升到 1.912 Hz。

因此，active-component loss 的结论是：

1. 单分量泄漏确实被 active loss 明显压低；
2. 如果把 mixed active checkpoint 直接替换双分量主模型，会伤害双分量；
3. 更合适的用法是让 active-count router 先判断分量数量：single active 或泄漏风险场景走 active-loss / single 分支，two active 仍走 simple_multicomponent_long。

16.4 P0 当前完成判断

到这里，P0 可以认为已经完成当前阶段的闭环：

- supervised quality selector 已经扩展到 42 维特征，并完成重新训练和独立评估；
- active-component loss 已经长训，并明确量化了 single active 收益与 two active 代价；
- jump 条件输入接口已经预留，后续可以接 Stage1 jump auxiliary；
- 质量头和 active loss 都没有盲目设为默认策略，而是保留为可控的诊断/路由分支。

当前默认建议仍然是：

- 单分量：继续优先使用 simple_single_component，active-loss 保守版作为泄漏抑制参考；
- 双分量：继续使用 simple_multicomponent_long；
- 多项式专项：保留 poly_multicomponent_refine 作为候选，不直接默认替换；
- quality selector：保留 stage2_quality_selector_p0_context，用于后续多专家融合和 hard-sample 诊断；
- 下一阶段再处理 P1：分段 refinement、多专家软融合、端到端分层解冻。

17. 2026-07-11 补充：P1 第一项，local_jump 分段 refinement

P0 完成之后，下一步按优先级先处理 P1 里的 local_jump 分段 refinement。原因是前面的评估已经说明，local_jump 的主要瓶颈不是 ICCD 层不能重构，也不是普通训练步数不够，而是跳变点附近的 IF 形状和普通连续 IF 的形状不一样。普通 refinement head 倾向于做全局平滑、小幅修正；这对 linear、quadratic、near_parallel 这类连续曲线是好事，但对 local_jump 会把真正应该快速变化的位置抹平，导致跳变点前后 IF 偏移。

17.1 本轮改动的核心原理

原来的 Stage2 refinement head 可以理解为一个统一修正器：

```
refined_if(t) = candidate_if(t) + delta_if(t)
```

这里的 delta_if(t) 由同一个 1D-CNN 生成。问题是它不知道哪些时间点是平稳段，哪些时间点是跳变段，所以只能学一个折中策略：既不能在跳变点改得太激进，也不能在平稳段完全不平滑。

本轮新增的分段 refinement 把这个过程拆成两个分支：

```
delta_if(t) =  
smooth_delta(t) * (1 - jump_mask(t))  
+ jump_delta(t) * jump_mask(t)  
  
refined_if(t) = candidate_if(t) + delta_if(t)
```

其中：

- `smooth_delta(t)` 负责普通平稳段，仍然保持较强平滑和较小修正幅度；
- `jump_delta(t)` 只在跳变区域起主要作用，允许比平稳段更大的局部修正；
- `jump_mask(t)` 由仿真数据中的 `jump_center` 和 `jump_valid` 生成，当前使用高斯形状的软 mask，而不是硬 0/1，这样跳变附近的过渡区也能被覆盖。

这一步没有解冻第一阶段 IF-Net。也就是说，Stage1 仍然只负责给出初始 IF 候选；Stage2 通过显式 `jump_mask` 条件输入学习“哪里应该允许局部快速修正”。这符合当前路线：先固定 IF-Net，把可微 ICCD 展开层和 refinement 结构训练稳定，再考虑端到端联合训练。

17.2 代码实现位置

本轮主要改动如下：

- `stage2_iccd/src/stage2_iccd/model.py`
- `Stage2ModelConfig` 新增 `refinement_mode` 和 `max_jump_refine_hz`；
- `IFRefinementHead` 新增 `segmented` 模式；
- `Stage2ICCDModel` 可以把 `refinement_extra` 传入 refinement head；
- 旧 checkpoint 仍可用，默认 `standard` 模式不改变原模型行为。
- `stage2_iccd/src/stage2_iccd/train_stage2.py`
- 新增 `build_refinement_extra(...)`；
- 训练和验证时会根据 batch 里的 `jump_center`、`jump_valid` 构造 `jump_mask`；
- 加入旧 checkpoint 到新结构的权重迁移：旧 refinement 第一层卷积权重复制到新 smooth 分支，新增条件通道用 0 初始化，jump 分支从头训练。
- `stage2_iccd/src/stage2_iccd/eval_scenarios.py`
- 分场景评估时也会根据 checkpoint config 自动构造 `refinement_extra`，保证训练和评估一致。
- `stage2_iccd/src/stage2_iccd/eval_active_routed_stage2.py`
- routed Stage2 评估路径同步支持 `refinement_extra`，后续把 `local_jump` 分支接入路由时不需要再改主体流程。
- `stage2_iccd/configs/local_jump_segmented_p1.yaml`
- 新增 `local_jump` 专项配置；
- 使用 `refinement_mode: segmented`；
- 使用 `refine_extra_channels: 2`，对应两个 IF 输出槽的 jump mask；
- 平稳段 `max_refine_hz` 为 26 Hz，跳变段 `max_jump_refine_hz` 为 70 Hz；
- 从 `stage2_iccd/runs/local_jump_frozen/latest.pt` 初始化，继续只训练 Stage2。

17.3 训练和独立评估结果

本轮训练时先遇到一个预期内的问题：旧的 `local_jump` checkpoint 第一层 refinement 卷积输入通道是 3，而新模型因为加入两个 jump-mask 条件通道，输入通道变成 5。直接加载会出现 shape mismatch。这个问题已经通过权重迁移解决：旧权重复制到已有通道，新通道置零，优化器状态不兼容时重新初始化优化器。

独立评估使用相同的 `local_jump` 鲁棒设置：

```
scenario = local_jump
batches = 64
batch_size = 6
SNR range = -4 dB ■ 22 dB
noise = white / colored / impulsive / trend ■■
```

模型	IF MAE / Hz	重构 SNR / dB	smooth	delta RMS / Hz	component L1
local_jump_frozen 基线	9.536	15.203	32.306	2.993	0.1365
local_jump_segmented_p1	8.086	15.788	14.075	1.819	0.1309

这个结果说明分段 refinement 的方向是有效的：

1. IF MAE 从 9.536 Hz 降到 8.086 Hz，约下降 15.2%；
2. 重构 SNR 从 15.203 dB 提升到 15.788 dB，说明 IF 修正没有牺牲重构稳定性；
3. smooth 从 32.306 降到 14.075，说明模型没有通过制造大量不规则抖动来追 IF；
4. delta RMS 从 2.993 Hz 降到 1.819 Hz，说明修正幅度整体更克制，但在 jump mask 区域更有针对性。

17.4 当前结论和边界

这一步可以认为已经完成 P1 第一项的首轮有效闭环：代码路径打通，旧 checkpoint 能迁移，训练能正常进行，独立评估优于 local_jump_frozen 基线。

但它还不能直接替代全部 Stage2 默认流程，原因有三点：

1. 这次只针对 local_jump 专项训练，没有证明它对 linear、quadratic、cubic、sinusoidal_fm、crossing 等类型都无回退；
2. 当前 jump_mask 来自仿真标签 jump_center，后续真实信号中需要由 Stage1 jump auxiliary 或其他事件检测模块提供；
3. crossing 的核心问题是身份一致性和轨迹交换，不能只靠 jump 分段修正解决。

因此当前推荐策略是：

- local_jump 专项分支：优先使用 local_jump_segmented_p1；
- 简单单分量：继续使用 simple_single_component；
- 简单双分量和 near_parallel：继续使用 simple_multicomponent_long 配合 active-count router；
- quality selector：继续作为诊断和后续多专家融合基础，不设为默认；
- 下一步 P1 优先级：先把 local_jump_segmented_p1 接入 routed Stage2 的候选分支，再推进多专家软融合；最后再做端到端分层解冻。

18. 2026-07-11 补充：P1 完整闭环结果

在完成 local_jump 分段 refinement 后，本轮继续把 P1 剩余四项也全部落地并验证。P1 的目标不是简单再训练一个更大的模型，而是补上第二阶段中几个会影响后续真实任务的结构能力：

1. 多专家候选不再只做全局平均，而是能按样本质量动态加权；
2. IF-Net 和可微 ICCD 之间具备分层解冻、联合微调的代码路径；
3. active-count router 从 1/2 分量扩展到 1/2/3 分量；
4. 训练流程具备在线难样本挖掘能力，让困难场景在后续 epoch 中被更多采样。

18.1 多专家 feature-attention 融合

原来的 `all_multiexpert` 虽然能同时加载 `balanced`、`polynomial`、`sinusoidal`、`local_jump` 等多个 IF-Net 专家，但融合方式仍然偏保守：主要依赖候选重构残差和全局可学习偏置。这样做比盲目平均好，但仍有一个问题：不同样本里“哪个专家可靠”并不固定。例如 `sinusoidal_fm` 中 `sinusoidal` 专家更有优势，`local_jump` 中 `jump` 专家更重要，而 `crossing` 里某个候选的重构残差可能短时很好但身份不稳定。

本轮在 `CandidateMixer` 中新增了 `candidate_fusion: feature_attention`。它不是直接让一个大网络重新预测 IF，而是给每个候选提取轻量质量特征：

- 候选自己的 ICCD 初步重构残差；
- 残差相对同批候选平均残差的比例；
- IF 均值、标准差、最大/最小范围；
- 一阶差分平均值，反映曲线斜率变化；
- 二阶差分峰值，反映局部弯折或突变；
- 候选与多专家候选中心的距离，反映它是不是离群候选。

这些特征进入一个小型 MLP，输出每个候选的额外质量分数，再和原来的残差门控一起形成 softmax 权重。它的好处是：Stage2 不再只问“这个候选能不能重构当前信号”，还会问“这个候选自身的 IF 形状是否合理、是否过度离群、是否局部过弯”。

对应配置为：

- `stage2_iccd/configs/all_multiexpert_feature_attention_p1.yaml`
- `checkpoint: stage2_iccd/runs/all_multiexpert_feature_attention_p1/latest.pt`

独立评估结果如下：

模型	aggregate IF MAE / Hz	aggregate SNR / dB	crossing IF MAE / Hz	local_jump IF MAE / Hz	tangent/overlap IF MAE / Hz
all_multiexpert 基线	13.822	14.222	33.082	12.723	14.544
feature_attention_p1	11.209	15.928	29.832	9.506	13.408

结论：feature-attention 是 P1 中最明确的全类型收益项。它没有完全解决 crossing，但把 aggregate IF MAE 降低约 18.9%，并同时提升重构 SNR，因此可以作为后续全类型多专家融合的主线。

18.2 在线难样本挖掘 OHEM

P1 的 OHEM 没有保存固定难样本，因为当前数据来自无限仿真，缓存具体波形反而会让训练过度记忆少数样本。本轮采用更轻的“场景级 OHEM”：

1. 训练过程中按 `print_every` 做验证；
2. 记录每个场景的验证 IF MAE；
3. 找出当前误差处于高分位的场景；
4. 在后续采样中提高这些场景的权重。

代码位置：

- `stage2_iccd/src/stage2_iccd/train_stage2.py`

- `evaluate(...)` 输出 `scenario_xxx_if_mae_hz`;
- `maybe_update_ohem_sampling(...)` 根据场景误差更新采样概率。

对应配置为：

- `stage2_iccd/configs/all_multiexpert_ohem_p1.yaml`
- `checkpoint: stage2_iccd/runs/all_multiexpert_ohem_p1/latest.pt`

独立评估结果如下：

模型	aggregate IF MAE / Hz	aggregate SNR / dB	crossing IF MAE / Hz	local_jump IF MAE / Hz	tangent/overlap IF MAE / Hz
feature_attention_p1	11.209	15.928	29.832	9.506	13.408
feature_attention + OHEM	10.743	16.575	28.562	8.678	12.055

结论：OHEM 有小幅但稳定的收益，尤其对 `local_jump`、`crossing` 和 `tangent_or_overlap` 这类困难场景更有帮助。因此当前全类型实验分支推荐使用 `all_multiexpert_ohem_p1`，但它仍不是简单单/双分量默认分支的替代品。

18.3 分层解冻联合训练

P1 中也实现了“先固定 IF-Net，重构稳定后再小学习率解冻 IF-Net 最后层”的代码路径。具体改动包括：

- `FrozenIFNetCandidateProvider` 新增 `trainable`、`unfreeze_last_decoders`、`unfreeze_head`;
- `train_stage2.make_optimizer(...)` 支持 Stage2 参数和 Stage1 参数使用不同学习率;
- 训练 checkpoint 可以保存和恢复可训练 provider 的状态;
- 默认情况下 provider 仍然完全冻结，旧流程不受影响。

本轮做了两个实验：

模型	解冻范围	aggregate IF MAE / Hz	aggregate SNR / dB	结论
simple_multicomponent_long 基线	不解冻	1.526	25.818	当前默认
unfreeze_p1	head + 最后 2 个 decoder	1.871	25.121	退化
unfreeze_head_p1	只解冻 head	1.730	25.026	比激进解冻稳，但仍差于冻结基线

结论：分层解冻的代码路径已经完成，但当前不进入默认训练策略。原因是 Stage2 已经能把简单场景修到很低误差，继续把梯度传回 IF-Net 容易破坏 Stage1 已有的稳定脊线输出。后续如果要重新推进端到端联合训练，应先在更困难的场景上使用更严格的门控，例如只对 `high-confidence` 且重构残差稳定下降的样本回传 Stage1 梯度，或者只解冻极少数归一化/输出层参数。

18.4 active-count 1/2/3 分量扩展

原 `active-count router` 只判断 1 分量或 2 分量。P1 中把它改为动态类别数，并新增峰值数量辅助特征：

- `active_count_names(num_classes)` 动态生成 `active_1`、`active_2`、`active_3`;
- checkpoint 保存 `active_count_names`，评估时按 checkpoint 自身类别加载;
- `compute_peak_count_features(...)` 统计 STFT 频率方向 top 峰比例，用来提示“当前像几条明显脊线”;

- routed Stage2 暂时采用兼容策略：active_1 走 single 分支，active_2/active_3 走 multi 分支。真正三分量 Stage2 需要后续单独训练 3 分量 IF-Net 和 3 分量 ICCD。

对应配置为：

- stage2_iccd/configs/active_count_123_peak_p1.yaml
- checkpoint: stage2_iccd/runs/active_count_123_peak_p1/latest.pt

独立评估结果：

指标	数值
aggregate accuracy	84.58%
active_1 accuracy	94.17%
active_2 accuracy	76.35%
active_3 accuracy	83.23%
confidence	70.48%

结论：1/2/3 active-count 原型已经可用，但不能直接替代当前 active_count_simple_near_parallel。主要短板是 active_2：两条接近平行或局部靠近的脊线容易被判断成 1 条宽脊线或 3 条峰。后续若要把三分量作为默认能力，需要补充 3 分量 Stage2 分支，并继续提升 active_2 与 near_parallel 的区分能力。

18.5 P1 完成判断

到目前为止，P1 可以认为已经完成工程闭环和效果筛选：

- 分段 local_jump refinement：有效，local_jump 专项分支可保留；
- 多专家 feature-attention：有效，是全类型融合的主线；
- OHem：有效，作为全类型训练增强保留；
- 分层解冻：代码路径完成，但当前效果不如冻结基线，不设为默认；
- active-count 1/2/3：原型完成，但不替代现有 1/2 router。

当前推荐组合更新为：

- 简单单分量：继续使用 simple_single_component；
- 简单双分量和 near_parallel：继续使用 simple_multicomponent_long + active_count_simple_near_parallel；
- local_jump：使用 local_jump_segmented_p1 作为专项分支；
- 全类型多专家实验：使用 all_multiexpert_ohem_p1；
- 三分量识别：保留 active_count_123_peak_p1，用于后续 3 分量 Stage2 准备；
- 端到端解冻：暂不默认启用，只保留 simple_multicomponent_unfreeze_head_p1 作为安全解冻参考。

下一阶段建议进入 P2 前先做两个收口动作：

1. 把 all_multiexpert_ohem_p1、local_jump_segmented_p1 和现有 active-count router 封装到统一推理 pipeline，明确每类样本走哪个分支；
2. 针对 crossing 单独增加身份一致性约束或轨迹匹配后处理，因为当前 P1 虽降低了 crossing 误差，但 crossing 仍然是全类型中最明显的短板。

19. 2026-07-11 补充：P1.5 稳定推理管线收口

P1 完成后，模型已经有多个有效分支，但还缺少一个稳定的“总入口”。如果每次评估都手动选择 checkpoint，后续进入 P2 时很容易出现两个问题：第一，不同实验使用的分支不一致，指标不可复现；第二，真实信号没有场景标签时，不知道应该调用单分量、双分量、local_jump 专项还是全专家分支。

因此 P1.5 的目标不是继续训练一个更大的网络，而是把 P1 中已经筛选出的有效模块封装为统一 pipeline，并把分支选择、置信度、top-2 候选和可视化输出固定下来。

19.1 P1.5 管线结构

新增代码如下：

- stage2_iccd/src/stage2_iccd/pipeline.py
- P15Stage2Pipeline：统一加载 active-count router、single 分支、multi 分支、local_jump 分支和 all-expert 分支；
- Stage2Branch：统一封装 Stage2 checkpoint 的加载、候选 IF 生成、refinement extra 构造和可微 ICCD 推理；
- 输出 branch、active_pred、active_confidence、active_probs、candidate_top2_weights、candidate_top2_indices；
- 额外输出 identity_stable_if_hz，用于 crossing 等场景的可视化连续轨迹展示。

- stage2_iccd/src/stage2_iccd/eval_p15_pipeline.py
- 用仿真信号批量评估 P1.5 pipeline；
- 输出每个场景的 IF MAE、重构 SNR、active 路由准确率、分支选择比例和 top-2 候选权重覆盖；
- 可选生成每个场景的 STFT + IF 叠加图。

- stage2_iccd/src/stage2_iccd/infer_p15_signal.py
- 支持输入 .npy 一维时域信号；
- 不需要真实 IF 标签；
- 自动输出预测 IF、活跃 IF、分支名、active 置信度、候选 top-2 权重和推理图片。

当前 P1.5 默认加载的分支为：

功能	checkpoint
active-count router	stage2_iccd/runs/active_count_simple_near_parallel/latest.pt
单分量分支	stage2_iccd/runs/simple_single_component/latest.pt
简单双分量/near_parallel 分支	stage2_iccd/runs/simple_multicomponent_long/latest.pt
local_jump 专项分支	stage2_iccd/runs/local_jump_segmented_p1/latest.pt
全类型困难场景分支	stage2_iccd/runs/all_multiexpert_ohem_p1/latest.pt

19.2 分支选择策略

在仿真评估中，如果有场景标签，P1.5 使用场景 hint 进行更稳定的策略选择：

- local_jump：直接走 local_jump_segmented_p1；
- crossing、sinusoidal_fm、tangent_or_overlap：走 all_multiexpert_ohem_p1；
- linear、quadratic、cubic、near_parallel：先由 active-count 判断单/双分量，再分别走 single 或 multi 分支。

对真实 .npy 信号，如果没有场景标签，则不使用 hint，pipeline 会退化为 active-count 路由：

- 判断为 1 个活跃分量：走 single；
- 判断为 2 个活跃分量：走 multi；
- 如果多分量置信度很低，可以进入 all-expert 作为保守备选。

这里曾尝试用 STFT 第二峰特征强行修正 near_parallel 的 active_2 误分，但评估发现该特征不能可靠区分“真实单分量宽脊线”和“双分量近并行脊线”。因此 P1.5 最终采用保守策略：不让二峰启发式覆盖高置信单分量判断，避免为了修 active_2 而重新引入单分量泄漏。

19.3 P1.5 全场景评估结果

运行命令如下：

```
$env:PYTHONPATH="stage2_iccd/src;ifnet_stage1/src"
.\.venv_ifnet\Scripts\python.exe -m stage2_iccd.eval_p15_pipeline --output-dir
stage2_iccd/runs/p15_pipeline/eval_default --batches 8 --batch-size 4 --plots-per-case 1 --snr-db-min -2
--snr-db-max 24 --noise-types-json "{white:0.55,colored:0.25,impulsive:0.10,trend:0.10}"
```

总体结果：

指标	数值
aggregate IF MAE	5.162 Hz
aggregate reconstruction SNR	20.578 dB
active route accuracy	96.09%
active route confidence	96.84%
top-2 candidate weight coverage	86.98%
single branch rate	25.78%
multi branch rate	24.02%
local_jump branch rate	12.50%
all_expert branch rate	37.70%

分场景 IF MAE 摘要：

场景	active=1 IF MAE / Hz	active=2 IF MAE / Hz	主要分支
linear	0.699	1.328	single / multi
quadratic	0.738	2.823	single / multi
cubic	1.120	3.666	single / multi
sinusoidal_fm	3.628	8.227	all_expert
crossing	9.463	25.996	all_expert
near_parallel	0.827	1.834	single / multi
local_jump	1.261	3.274	local_jump
tangent_or_overlap	7.574	10.127	all_expert

这个结果说明 P1.5 已经达到“统一入口”和“稳定基线”的目的：简单场景和 local_jump 的表现稳定，near_parallel 在保守路由下没有被二峰启发式破坏；但 crossing 仍然是最明显短板，尤其是双分量 crossing 的 IF MAE 仍接近 26 Hz。

19.4 .npy 信号推理验证

P1.5 还新增了不依赖真实标签的单信号入口。测试时构造了一个一维 AM-FM 测试信号并保存为 tmp/p15_test_signal.npy，运行：

```
$env:PYTHONPATH="stage2_iccd/src;ifnet_stage1/src"
.\.venv_ifnet\Scripts\python.exe -m stage2_iccd.infer_p15_signal --input-ntp tmp\p15_test_signal.npy --output-dir
stage2_iccd/runs/p15_pipeline/infer_smoke --fs 1024
```

输出结果：

- 分支：single
- active-count 预测：1 个活跃分量
- active 置信度：99.46%
- 输出完整 IF：stage2_iccd/runs/p15_pipeline/infer_smoke/p15_if_hz.npy
- 输出活跃 IF：stage2_iccd/runs/p15_pipeline/infer_smoke/p15_active_if_hz.npy
- 输出图片：stage2_iccd/runs/p15_pipeline/infer_smoke/p15_inference.png

这说明后续真实信号或外部图像反推得到的时域信号，可以先通过 .npy 入口进入 P1.5 pipeline，再统一生成 STFT 叠加图和 IF 曲线。

19.5 P1.5 完成判断与进入 P2 条件

P1.5 可以认为已经完成以下闭环：

- 已有 P1 分支被封装为统一 pipeline；
- 仿真批量评估、分支比例统计、top-2 候选统计和图片输出已固定；
- .npy 真实/外部信号推理入口已打通；
- P1.5 默认组合已经明确，可以作为 P2 的输入基线。

但进入 P2 前需要保留两个风险判断：

1. top-2 candidate weight coverage = 86.98%，距离之前希望稳定超过 88% 还差约 1 个百分点。这个问题主要来自 all-expert 分支在 sinusoidal、crossing、tangent_or_overlap 上候选权重较分散。
2. crossing 仍然没有真正解决。P1.5 通过 all-expert 分支降低了部分误差，但双分量 crossing 的重构和身份一致性仍然偏弱。

因此，下一步可以进入 P2，但 P2 的第一件事不应是全面扩展真实数据或模型蒸馏，而应先围绕 crossing 做结构性修复：加入身份一致性损失、轨迹连续匹配、crossing 专项候选融合，或者在可微 ICCD 展开层中加入交叉点附近的局部重构约束。完成这个补强后，再推进 P2 的真实数据域适应和三分量扩展会更稳。

20. 2026-07-11 补充：P2 工程闭环

P2 的目标不是只继续调参，而是把“可微 ICCD + IF-Net”的第二阶段从单一实验扩展成可以支撑后续真实信号、三分量、物理正则和推理加速的工程体系。本轮 P2 按照 P1.5 暴露的问题推进：先补 crossing，再补三分量验证、真实信号域差诊断、Tiny 模型蒸馏和统一 HTML 报告。

20.1 P2 物理正则与 crossing 身份约束

新增代码位置：

- stage2_iccd/src/stage2_iccd/losses.py
- if_third_derivative(...): 约束 IF 的三阶变化，用于多项式/平滑轨迹；

- `crossing_identity_loss(...)`: 在两条 IF 接近时, 不固定频率上下顺序, 而是约束每个输出槽的速度连续性, 避免 crossing 附近突然换身份;
- `min_gap_barrier(...)`: 弱约束过近分量, 主要用于三分量或非 crossing 场景;
- `sinusoidal_curvature_consistency(...)`: 给正弦调频类轨迹一个“曲率能量不过度局部爆发”的弱先验。
- `stage2_iccd/src/stage2_iccd/train_stage2.py`
- `compute_loss(...)` 已接入上述正则;
- 默认权重为 0, 不影响 P1/P1.5 旧配置;
- 只有在 config 的 `train.loss` 中显式设置 `third_derivative`、`crossing_identity` 等权重时才启用。

新增 crossing 专项配置:

- `stage2_iccd/configs/crossing_identity_p2.yaml`
- `checkpoint: stage2_iccd/runs/crossing_identity_p2/latest.pt`

该配置从 `all_multiexpert_ohem_p1` 继续训练, 只采样 crossing 场景, 并加入:

- `third_derivative: 0.00025`
- `crossing_identity: 0.003`
- `crossing_gap_sigma_hz: 30.0`

独立 crossing 评估结果:

模型	crossing IF MAE / Hz	crossing SNR / dB	top-2 覆盖
P1.5 all-expert crossing 路由	27.749	4.608	78.44%
P2 crossing_identity_p2 路由	23.941	4.821	81.55%

结论: P2 crossing 专项分支有效, IF MAE 下降约 13.7%, top-2 候选覆盖也有提升。但 crossing 仍未彻底解决, 尤其是身份稳定后处理 `post_identity_if_mae_hz` 反而可能变差, 说明后处理不能替代训练期的身份建模。

20.2 P2 全场景 pipeline 结果

P2 pipeline 在 P1.5 基础上显式接入 crossing 专项分支:

```
$env:PYTHONPATH="stage2_iccd/src;ifnet_stage1/src"
.\venv_ifnet\Scripts\python.exe -m stage2_iccd.eval_p15_pipeline --output-dir
stage2_iccd/runs/p2_pipeline/eval_default --batches 6 --batch-size 4 --plots-per-case 1 --crossing-checkpoint
stage2_iccd/runs/crossing_identity_p2/latest.pt --snr-db-min -2 --snr-db-max 24 --noise-types-json
"{white:0.55,colored:0.25,impulsive:0.10,trend:0.10}"
```

总体结果:

指标	P1.5	P2
aggregate IF MAE	5.162 Hz	4.506 Hz
aggregate reconstruction SNR	20.578 dB	20.775 dB
active route accuracy	96.09%	96.35%
top-2 candidate coverage	86.98%	87.15%
crossing active=2 IF MAE	25.996 Hz	21.349 Hz

分支比例:

分支	比例
single	25.52%
multi	24.22%
local_jump	12.50%
crossing	12.50%
all_expert	25.26%

结论：P2 相比 P1.5 有明确收益，尤其是 crossing 双分量从 25.996 Hz 降到 21.349 Hz。但 top-2 覆盖率仍只有 87.15%，没有稳定超过 88%，说明多专家候选融合仍是后续可以继续提高的点。

20.3 三分量 Stage2 / ICCD 验证

新增配置：

- stage2_iccd/configs/three_component_oracle_p2.yaml
- checkpoint: stage2_iccd/runs/three_component_oracle_p2/latest.pt

该配置使用 num_components: 3 和 oracle_perturbed 候选 IF。这里的目标不是宣称真实三分量 IF-Net 已经完成，而是验证 Stage2 的可微 ICCD、active mask、component matching 和三分量 loss 能否稳定工作。

独立评估结果：

指标	数值
aggregate IF MAE	6.458 Hz
aggregate SNR	15.442 dB
linear IF MAE	6.524 Hz
quadratic IF MAE	6.553 Hz
cubic IF MAE	6.380 Hz
near_parallel IF MAE	6.267 Hz
crossing IF MAE	6.565 Hz

结论：三分量 ICCD 展开层和损失函数已经跑通。下一步若要真正支持三分量真实推理，还需要训练三分量 IF-Net 或三分量候选生成器，而不是继续依赖 oracle 候选。

20.4 真实/外部信号域差诊断

新增代码：

- stage2_iccd/src/stage2_iccd/domain_adaptation.py

功能：

1. 读取一个目录下的 .npy 一维时域信号；
2. 计算多尺度 STFT 特征统计；
3. 与仿真训练分布的 STFT 特征做差；
4. 输出 feature_l1、feature_l2 和均值统计。

测试命令：

```
$env:PYTHONPATH="stage2_iccd/src;ifnet_stage1/src"
.\.venv_ifnet\Scripts\python.exe -m stage2_iccd.domain_adaptation --npz-dir tmp --output-json
```

```
stage2_iccd/runs/p2_domain/domain_summary.json --max-files 8
```

测试输出：

指标	数值
num_files	1
feature_l2	0.524
feature_l1	0.104

结论：真实/外部信号进入 P2 前，现在已经有一个量化“和仿真训练分布差多远”的入口。它不更新 Stage1，也不直接微调模型；用途是在做 Stage2-only 域适应前先判断输入信号是否明显偏离当前仿真域。

20.5 Tiny-IF-Net 蒸馏

新增代码：

- stage2_iccd/src/stage2_iccd/train_tiny_distill.py

训练方式：

- teacher: P1.5/P2 routed Stage2 pipeline;
- student: 轻量 TinyIFNet;
- 输入: 多尺度 STFT 特征;
- 监督: teacher 输出的 identity_stable_if_hz;
- 目的: 把多分支 Stage2 推理蒸馏为一个更快的 IF 估计器。

测试命令：

```
$env:PYTHONPATH="stage2_iccd/src;ifnet_stage1/src"
.\.venv_ifnet\Scripts\python.exe -m stage2_iccd.train_tiny_distill --run-dir
stage2_iccd/runs/tiny_ifnet_distill_p2 --steps 80 --batch-size 6
```

短训练结果：

step	teacher-student IF MAE
10	90.204 Hz
30	32.301 Hz
50	26.296 Hz
80	28.471 Hz

结论：蒸馏入口已经跑通，学生模型能快速下降，但 80 步结果仍明显不够好，不能作为默认推理替代。后续若要做部署加速，需要更长训练、更强 student、场景均衡采样，以及同时监督 active-count 和 IF。

20.6 HTML 报告

新增代码：

- stage2_iccd/src/stage2_iccd/p2_report.py

功能：

- 读取 eval_p15_pipeline.py、eval_scenarios.py 或 routed eval 的 metrics JSON;
- 自动生成 HTML 表格;
- 自动嵌入 overview 和各场景图片;
- 自动提示 top-2 覆盖不足、crossing 风险等问题。

已生成报告:

- stage2_iccd/runs/p2_pipeline/eval_default/p2_report.html
- stage2_iccd/runs/p2_pipeline/eval_crossing_routed/p2_report.html

20.7 P2 完成判断

到目前为止, P2 可以认为完成了工程闭环:

- crossing 专项分支完成并接入统一 pipeline;
- 物理 IF 正则完整接入训练 loss;
- 三分量 Stage2/ICCD 路径跑通;
- .npy 真实/外部信号域差诊断入口跑通;
- Tiny-IF-Net 蒸馏入口跑通;
- HTML 报告入口跑通;
- README 和 PDF 总结已同步。

但 P2 的实验结论必须保持克制:

1. crossing 有改善, 但仍是最大误差来源;
2. top-2 candidate coverage 从 86.98% 提升到 87.15%, 仍未稳定超过 88%;
3. 三分量目前是 oracle 候选验证, 不是真实三分量 IF-Net 推理;
4. Tiny 蒸馏只是接口完成, 还不能替代多分支 Stage2;
5. 域适应目前是诊断入口, 不是已经完成真实数据微调。

因此, 当前可以进入下一阶段的真实信号实验, 但推荐路线是: 先用 P2 pipeline 处理真实 .npy 信号并生成 HTML 报告, 再根据 domain gap 决定是否只微调 Stage2; 不要直接解冻 Stage1, 也不要马上把 Tiny 模型作为主推理模型。